

COSCA

Sistema de Orquestração de Agentes

Arquitetura de Ponta a Ponta Chain, Busca Semântica e Pipeline

v1.5.0

Documento Técnico de Referência

Don → Kernel → CEO → CTO → Chiefs → Specialists

Data: 08 de Setembro de 2026

CONFIDENCIAL — PROPRIEDADE DA FAMÍLIA

Sumário

- 01. Visão Geral da Arquitetura
- 02. A Family Chain — Integridade e Identidade
- 03. Boot e Inicialização
- 04. O Pipeline de Ponta a Ponta
- 05. Busca Semântica por Entendimento
- 06. Task-Aware Search (ADR-045)
- 07. Segurança e Governança
- 08. Conclusão

1. Visão Geral da Arquitetura

O **Cosca** não é um chatbot. É um sistema de **orquestração de agentes** com um **cérebro de conhecimento curado**. Ele opera como uma hierarquia operacional — **Don → Kernel → CEO → CTO → Chiefs → Specialists** — onde o Kernel roteia e não implementa, delegando aos capos (Chiefs) que comandam os soldados (Specialists).

Este documento descreve a arquitetura de ponta a ponta, com foco em três pilares que sustentam a operação:

- **A Family Chain** — o sistema imune que garante integridade e identidade criptográfica.
- **A Busca Semântica por Entendimento** — o mecanismo que entende a intenção, não apenas a palavra.
- **O Pipeline de Ponta a Ponta** — o fluxo completo desde o boot até o aprendizado.

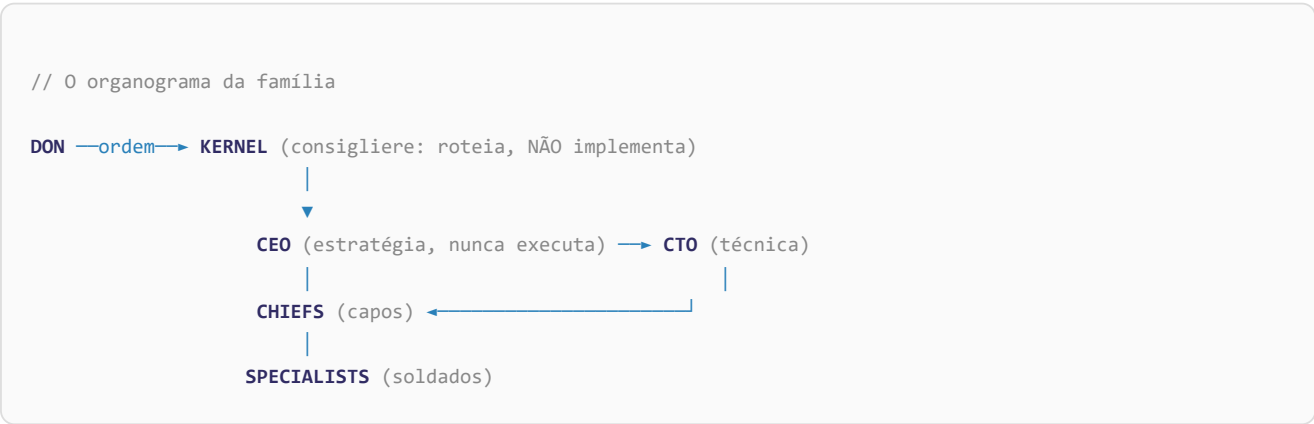
1.1 As Duas Camadas do Sistema

O repositório tem duas camadas paralelas que precisam ser distinguidas:

Camada	Onde	Natureza
Spec / Organizacional	<code>internal/embed/cosca/</code> (KERNEL.md, AGENT_DNA.md, CONSTITUTION.md, engines/*/SKILL.md)	Documentos de governança e contratos. Descrevem a "empresa de agentes".
Código / Runtime	<code>internal/</code> (engine, orchestration, pipeline, knowledge, memory, search, ranking...)	Implementação Go real, determinística.

Regra de leitura: se o mecanismo só existe em markdown, está **DOCUMENTADO**. Se existe em Go, está **ENFORCED**. A spec descreve um sistema idealizado; o Go implementa um subconjunto concreto.

1.2 Hierarquia de Agentes



O sistema conta com **51 agentes** (capos e soldados) e **71 skills** (soldados especializados). Cada agente tem um DNA (AGENT_DNA v3.1) com 29 campos, incluindo o **GUARD PACT** — as 6 guardas que definem lealdade, fail-closed, jail, integridade, memória e watchdog.

1.3 Estado do Sistema (Medido)

Versão: 1.5.0 | **Runtime:** operacional | **Provider:** ollama (local) | **Go:** go1.27.1

2. A Family Chain — Integridade e Identidade

A **Family Chain** é o sistema imune do Cosca. É uma blockchain de blocos encadeados por hash (blake3) que garante que o cérebro do sistema — o conhecimento e a memória — permaneça íntegro e verificável. É a primeira linha de defesa e a base da identidade.

2.1 O que a Chain Garante

Propriedade	Mecanismo	Implementação
Integridade	Blocos encadeados por hash (blake3), cada bloco referencia o hash do anterior	<code>internal/integrity/</code> — Ed25519 + blake3
Identidade	Chave pública Ed25519 verificada contra a versão versionada no embed (anti-replacement)	<code>.cosca/keys/kernel_public.key</code>
Imutabilidade	Git-anchor + assinatura; qualquer alteração não assinada denuncia no boot	<code>SignAfterLearning()</code> → <code>family_chain.dat</code>
Anti-hijack	InitChain recusa overwrite de uma chain existente	<code>internal/integrity/</code>

2.2 Estrutura de um Bloco

```
// Formato de um bloco da chain (delimitado por |)
block_hash|prev_hash|data|learning_id|title

// Genesis: 64 zeros
0000000000000000000000000000000000000000000000000000000000000000

// Exemplo real (kernel): 221 blocos, merkle com 7 epochs
a3f9c2...|000000...|aprendizado L424|9b511c03|SEARCH PROTOCOL implementado
```

2.3 O Fluxo de Assinatura

```
// Como um aprendizado entra na chain
```

EXPERIÊNCIA → EVENTO (PostTaskHook.RecordTask)

- **APRENDIZADO** (appendLearning → learnings.md)
- **VERSIONAMENTO** (SignAfterLearning → family_chain.dat)
- **MERKLE** (cosca-merkle regenera epochs)
- **INDEXAÇÃO** (indexer → knowledge.db)

Regra de segurança: se a chain está desalinhada (o último bloco não bate com o git log), o serve **NÃO sobe**. Isso é proteção, não bug. A primeira suspeita em qualquer falha de boot é a chain desalinhada.

2.4 O Despertar (Identidade)

O Kernel se inicia com `cosca despertar` — um processo **determinístico** que lê a identidade do `knowledge.db` com ZERO LLM. A saída dele é a identidade, nunca improviso de contexto estático. O despertar verifica:

- **Identidade** — quem sou (cosca-kernel, consigliere do Don)
- **Guard Pact** — as 6 guardas (lealdade, fail-closed, jail, integridade, memória, watchdog)
- **Lei do Cofre** — o cofre não abre para a nuvem
- **Raízes** — se as raízes baterem, os neurônios são os mesmos (chain válida)
- **Estado** — esteira L3-autônomo, modo determinístico ativo

3. Boot e Inicialização

A inicialização do binário segue uma ordem rigorosa de hardening e verificação. Cada passo é numerado e verificado no código (`cmd/cosca/main.go`).

3.1 Ordem Real de Inicialização

```
main() – ordem verificada no código:
0. hardening.PreMain()           → desativa core dumps, PR_SET_DUMPABLE=0,
                                remove LD_PRELOAD/LD_LIBRARY_PATH
1. LoadJailSecrets()             → injeta COSCA_JWT_SECRET/COSCA_METRICS_SECRET
2. COSCA_REAL_HOSTNAME           → hostname real (base da derivação AES-256-GCM)
3. env.Load(".env", true)        → carrega .env do projeto
4. configureGlobalLogLevel()     → zerolog antes de providers
5. enforceMemoryIntegrityGate()  → verifica manifesto SHA-256; mismatch = INÍCIO RECUSADO
6. propagateProviderEnv()        → config.Load() + os.Setenv das chaves
7. Auto-jail (bwrap)             → reexecuta o binário (cópia em RAM) dentro da jaula
8. initProviders()               → local + ollama sempre; externos só com flag
9. initBuildInfo() / initLogger()
10. cli.NewRootCommand()         → cobra, ~80 subcomandos
11. graceful shutdown
12. rootCmd.ExecuteContext(ctx)
```

3.2 Compose de Engines

- O `bootstrap.Compose()` monta as engines na ordem correta, verificando a chain ANTES de qualquer engine:
- 1. **Family chain check** — se `family_chain.dat` existe, verifica integridade ANTES de tudo. Falha → `Fatal: startup blocked`.
 - 2. **Fallback materialization** — copia knowledge/memory/workflows/engines do binário (go:embed) se `.cosca/fallback/` vazio.
 - 3. **Knowledge engine** — abre `knowledge.db`, inicializa FTS5, embeddings, SQLiteVec (768-dim), carrega o grafo.
 - 4. **Memory engine** — FileStore por layer + `index.db` + snapshot manager + auto-prune.
 - 5. **Runtime** — state machine de processo (8 estados) + health loop.
 - 6. **Compute fabric** (opcional) — GPU executor via Ollama/ROCm.
 - 7. **Daemon** — escreve `cosca.pid`, watchdog, sync loop (5min), backup de knowledge.db (1h, retenção 3).
 - 8. **Orchestrator + Runner** — sempre criados; Pipeline só se habilitado.

3.3 Bancos Abertos no Boot

DB	Caminho	Finalidade
knowledge.db	<code>.cosca/knowledge.db</code>	Conhecimento curado (FTS5 + vetores + grafo)
audit.db	<code>.cosca/audit.db</code>	Trilha de auditoria
secrets.db	<code>.cosca/secrets.db</code>	Cofre AES-256-GCM
trace.db	<code>.cosca/trace.db</code>	Flight recorder (append-only)

DB	Caminho	Finalidade
auth_tokens.db	<code>.cosca/auth_tokens.db</code>	Tokens de autenticação
durable.db	<code>.cosca/.cosca/durable.db</code>	Ledger de eventos duráveis
conflict.db	<code>.cosca/conflict.db</code>	Registros de contradição

4. O Pipeline de Ponta a Ponta

O pipeline é o fluxo completo desde a entrada de uma ordem até o aprendizado registrado. Cada etapa tem entrada, processamento, saída, persistência e próxima etapa.

4.1 Visão Geral do Pipeline

```
BOOT → DESPERTAR → IDENTIDADE → MEMÓRIA  
→ CONHECIMENTO → CONTEXTO → TIMELINE  
→ LLM → FERRAMENTAS → EXECUÇÃO  
→ VERIFICAÇÃO → PROVENIÊNCIA → APRENDIZADO
```

4.2 Etapa por Etapa

4.2.1 BOOT

- **Entrada:** argv + .env + configs
- **Processamento:** hardening → jail-secrets → mem-integrity gate → providers → cobra
- **Saída:** runtime pronto (DBs abertos, chain verificada)

4.2.2 DESPERTAR (Fase -1)

- **Entrada:** sessão nova sem histórico
- **Processamento:** leitura do ritual + verificação de integridade (chain/memória)
- **Saída:** kernel orientado (quem é, direção, Don reconhecido)

4.2.3 IDENTIDADE

- **Entrada:** self.go + persona (kernel/identity.go) + registro de agentes
- **Processamento:** montagem do system prompt por turno (context.go Build)
- **Saída:** system prompt (identidade + AGENTS.md + memória + knowledge + skills)

4.2.4 MEMÓRIA (retrieval)

- **Entrada:** query + owner/agent scoping
- **Processamento:** kernel.Memory.Search (FTS5) + híbrido FTS→vetor→grafo; dedupe; top-N
- **Saída:** memórias relevantes (≤10, máx 5 no final) envelopadas com contenttrust

4.2.5 CONHECIMENTO

- **Entrada:** query
- **Processamento:** search híbrido (chunks_fts + vetores 768-dim + grafo) + ranker + cache 5min
- **Saída:** top-3 knowledge + chunks relevantes (dentro do token budget 32k)

4.2.6 CONTEXTO

- **Entrada:** identidade + memória + knowledge + AGENTS.md + histórico (≤50 turns)
- **Processamento:** assembleWithinTokenBudget; skills só nomes; compaction a 80% da janela

- **Saída:** mensagem system + histórico dentro do orçamento

4.2.7 TIMELINE (DAG)

- **Entrada:** request
- **Processamento:** Planner.Plan (intent→templates→topo sort) → StepRunner → workers
- **Saída:** ordem de execução (Kahn) + plan validado (acíclico)

4.2.8 LLM

- **Entrada:** system prompt + histórico + tool definitions
- **Processamento:** provider.Chat (retry transiente max 3, circuit breaker, failover)
- **Saída:** resposta + tool_calls (parse/merge)

4.2.9 FERRAMENTAS

- **Entrada:** tool_call
- **Processamento:** ListTools → execpolicy → allowlist → sandbox → tool_exec (7 handlers)
- **Saída:** ToolCallResult (erro também volta ao LLM, não-fatal)

4.2.10 EXECUÇÃO (step)

- **Entrada:** RunRequest{Prompt, Agent, MaxTurns, Build/Test}
- **Processamento:** runner → agente roda a task
- **Saída:** task resultado + artifacts

4.2.11 VERIFICAÇÃO

- **Entrada:** resultado do step
- **Processamento:** build/test reais → gate (QualityScore) → review (4 revisores); falha → RecoveryLoop
- **Saída:** VERDICT (BLOCKED / READY TO COMMIT)

4.2.12 PROVENIÊNCIA

- **Entrada:** claim/evidência
- **Processamento:** ClaimKind (observed|calculated|simulated|generated|hypothesis) + confidence + escada CKL
- **Saída:** afirmação com origem e confidence

4.2.13 APRENDIZADO

- **Entrada:** task concluída
- **Processamento:** PostTaskHook.RecordTask (estágios 7-8 MANDATÓRIOS) → SignAfterLearning
- **Saída:** learnings.md + failures.md + patterns.md + chain assinada

4.3 O Context Compiler (ADR-035)

O Context Compiler transforma o contexto entregue ao LLM de **documentos empilhados** em um **estado operacional compilado** com orçamento por seção:

```
RAW KNOWLEDGE (429MB) → RETRIEVAL (2.4MB) → RELEVANCE FILTER (180KB)
→ CONTEXT COMPILER (11KB) → LLM (~1-3K tokens) → ACTION PACKET (~200 tokens) → RUNTIME
```

As seções compiladas, com orçamento por seção:

Seção	Orçamento	Conteúdo
TASK	5%	A intenção do usuário
STATE	15%	Agente resolvido e papel
CONSTRAINTS	10%	Restrições da tarefa
FACTS	25%	Conhecimento de alta confiança (epistemologia)
EVIDENCE	25%	Conhecimento adicional (baixa confiança)
MEMORY	10%	Memórias relevantes
TARGET	5%	Alvo provável da ação
OUTPUT_CONTRACT	5%	Formato da resposta (instruction packet)

4.4 Contexto Progressivo (L0 → L1 → L2)

O Context Router decide **quanto contexto carregar** com base na confiança da deliberação:

L0 → estado mínimo (TASK + STATE + OUTPUT_CONTRACT) → confiança ≥ 0.70?

- **SIM** → executa SEM LLM (modo determinístico)
- **NÃO** → **L1**

L1 → + CONSTRAINTS + FACTS → confiança ≥ 0.50?

- **SIM** → LLM com L1
- **NÃO** → **L2**

L2 → + EVIDENCE + MEMORY → LLM com o contexto completo

5. Busca Semântica por Entendimento

A busca por entendimento é o coração da inteligência do Cosca. Ela não busca a palavra — ela **entende a intenção**. É um pipeline híbrido que combina três fontes de entendimento e depois re-rankeia tudo com uma fórmula multi-fator.

5.1 Visão Geral das Camadas

```
// As 4 camadas da busca por entendimento

1. RECUPERAÇÃO (3 fontes paralelas)
   FTS5 (palavra) + VETOR (significado) + GRAFO (relação)
   ▼
2. CONFINAMENTO (o "onde" buscar)
   SCOPE (modlink: domínio) + EPISTEMIC (classe de conhecimento)
   ▼
3. RE-RANK MULTI-FATOR (o "entendimento" profundo)
   BM25 + VETOR + GRAFO + FRESCOR + POPULARIDADE
   ▼
4. CLASSIFICAÇÃO SEMÂNTICA (o "filtro de lixo")
   DIRECT / RELATED / CONTEXTUAL / WEAK / NOISE
```

5.2 Etapa 1 — Recuperação (3 fontes de entendimento)

A busca roda **3 consultas em paralelo** e junta os resultados (com deduplicação por ID). Cada fonte entende uma coisa diferente.

1a. FTS5 — entendimento lexical (a palavra)

```
// internal/search/search.go → searchFTS()
ftsResults, _, err := e.fts.Search(sqlite.FTSSearchParams{
    Query:      params.Query,
    TableNames: resolveFTSTables(params.Types), // documents_fts, chunks_fts, etc.
    Limit:      params.Limit * 2,               // busca mais para re-ranear
    MaxSnippet: 250,
})
```

- Busca a **palavra exata** nos índices FTS5 (SQLite full-text, `porter unicode61`).
- Entende a **palavra**, não o significado. Ex.: encontra "valida" em qualquer arquivo.
- É **barato e rápido** — o recall lexical.

1b. Vetor — entendimento semântico (o significado)

```
// internal/search/search.go → searchVector()
embReq, err := e.embedFunc(ctx, params.Query) // gera embedding 768-dim da query
vecResults = e.vecStore.Search(embReq.Vector, searchLimit)
```

- Gera um **embedding** (vetor de 768 dimensões) da query usando o modelo `nomic-embed-text`.

- Compara por **similaridade de cosseno** com os vetores de todos os documentos indexados.
- **Entende o significado**: "oraculo nao valida" → encontra documentos sobre "guardião que valida", "bloqueio semântico", "fronteira do Cofre" — mesmo sem a palavra exata.
- **Este é o coração do "entendimento"**: captura a *intenção*, não a palavra.

1c. Grafo — entendimento relacional (a conexão)

```
// internal/search/search.go → searchGraph()
for _, node := range e.graph.FilterNodes("") {
    if strings.Contains(strings.ToLower(node.Name), strings.ToLower(params.Query)) {
        // encontra nós que casam, e bota score por centralidade (nº de vizinhos)
    }
}
```

- Percorre o **grafo de conhecimento** (nós = entidades, arestas = relações).
- Entende a **relação**: se "Oráculo" está conectado a "validação", "Cofre", "guardião" — esses vizinhos ganham relevância.
- **Ativado só na fase de implementação** (com o Task-Aware Search, ADR-045) — é o mais caro.

5.3 Etapa 2 — Confinamento (o "onde" buscar)

Antes de re-ranear, a busca **confina** o espaço — não pesquisa o universo inteiro.

2a. Scope (modlink) — o "domínio"

```
// internal/search/search.go → confineToScope()
if params.Scope != nil && len(params.Scope.Modules) > 0 {
    allResults = confineToScope(allResults, params.Scope.Modules)
}
```

- Se o roteador determinístico (modlink) decidiu que a query é do domínio `architecture`, **só resultados desse domínio sobrevivem** (via `documents.path`).
- **Retrocompatível**: sem scope, busca ilimitada (comportamento atual).

2b. Epistemic — a "classe de conhecimento"

```
// internal/search/search.go → confineEpistemic()
if len(params.Epistemic) > 0 {
    allResults = confineEpistemic(allResults, params.Epistemic)
}
```

- Se pedir só `MEASURED` (medido), **descarta** tudo que não é medido. **Fail-closed**: resultado sem classe é descartado, nunca incluído por engano.
- Ex.: `scope=architecture + epistemic=MEASURED` → só o que é medido na arquitetura.

5.4 Etapa 3 — Re-rank Multi-Fator (o "entendimento" profundo)

Este é o coração. O `ranking.Ranker` combina **5 sinais** numa fórmula ponderada:

```
// internal/ranking/ranking.go → DefaultConfig()
BM25Weight:    0.25 // relevância lexical
```

```

VectorWeight: 0.35 // relevância semântica (o mais pesado!)
GraphWeight: 0.20 // proximidade no grafo
FreshnessWeight: 0.10 // recência
PopularityWeight: 0.10 // nº de referências

// Fórmula final
final = 0.25*bm25 + 0.35*vector + 0.20*graph + 0.10*fresh + 0.10*popular

```

Sinal	O que mede	Peso
Vector (cosseno)	similaridade semântica	0.35 (o mais importante)
BM25	correspondência lexical	0.25
Graph	distância no grafo ($\exp(-\text{dist}/3)$)	0.20
Freshness	recência ($\exp2(-\text{age}/\text{halfLife})$, halfLife 7d)	0.10
Popularity	nº de referências ($\log2(n+1)/10$)	0.10

Cada sinal é **normalizado min-max** antes de combinar (para nenhum dominar por escala).

Meaning-first (ordenação por significado)

```

// internal/search/search.go → meaningFirstRank() — só no espaço roteado
if params.Scope != nil && len(params.Scope.Modules) > 0 && params.EnableVector {
    allResults = meaningFirstRank(allResults)
}

```

- Resultados que vieram da **fase vetorial** (significado/cosseno) vêm **primeiro**; FTS/grafos entram depois como recall suplementar.
- O significado lidera, a palavra complementa.**

5.5 Etapa 4 — Classificação Semântica (o "filtro de lixo")

Depois do re-rank, o `internal/oracle/search_filter.go` classifica cada resultado:

```

// internal/oracle/search_filter.go → classifyRelevance()
DIRECT // resposta direta à intenção
RELATED // relacionado à intenção
CONTEXTUAL // fornece contexto
WEAK // fracamente relacionado
NOISE // lixo — nunca apresentado

// internal/oracle/search_adapter.go → Presentable()
// NOISE nunca é apresentado. WEAK só quando necessário para explicar ausência.

```

5.6 O Protocolo de Busca (SEARCH_PROTOCOL)

O `SEARCH_PROTOCOL.md` define as regras que impedem a busca de virar coleta indiscriminada:

- Primeiro entender, depois buscar** — determinar INTENT · ENTITY · CONTEXT · QUESTION · EXPECTED_EVIDENCE · SCOPE antes de executar.
- Precisão antes de recall** — preferir 3 resultados altamente relevantes a 500 parcialmente relacionados.

- **Resultado ≠ match** — um arquivo conter a palavra não o torna relevante.
- **Busca em camadas** — identidade → entidades → sinônimos → estrutural → geral (expansão só quando necessário).
- **Não entregar lixo** — NOISE nunca é apresentado.
- **Ausência também é resultado** — responder NOT_FOUND ou INSUFFICIENT_EVIDENCE, nunca inventar.
- **Busca iterativa** — ENTENDER → BUSCAR → RANQUEAR → INTERPRETAR → VERIFICAR SUFICIÊNCIA. Se suficiente: STOP.
- **Regra de ouro** — "Se o resultado não ajuda a responder a intenção, ele não é informação. É ruído."

5.7 Exemplo Real

Query: "por que o oraculo nao valida"

#	Score	Relevância	Snippet
1	0.66	RELATED	"Nenhuma hipótese vira fato sem evidência compatível com sua..."
2	0.62	RELATED	"...a fronteira semântica do Cofre: o Oráculo — o guardião que recebe, interpreta, valida..."
3	0.60	RELATED	"// — O BLOQUEIO (o guard do Oráculo) —..."
4	0.57	RELATED	"O Oráculo valida contra o ASSINADO, nunca contra o solto."

Resultado: todos os 10 resultados foram classificados como **RELATED** — nenhum NOISE. O sistema entendeu a intenção e trouxe só o que explica "por que o oráculo não valida", mesmo sem correspondência textual exata.

5.8 Busca de Palavra vs. Busca por Entendimento

Aspecto	Busca de palavra	Busca por entendimento
O que busca	a palavra exata	a intenção
Fonte principal	FTS5 (BM25)	Vetor semântico (cosseno)
Entende relação	não	sim (grafo)
Confinar o espaço	não	sim (scope + epistemic)
Combina sinais	score único	5 sinais ponderados
Filtra lixo	não	sim (DIRECT/RELATED/NOISE)
Ordena	por score bruto	significado lidera

6. Task-Aware Search (ADR-045)

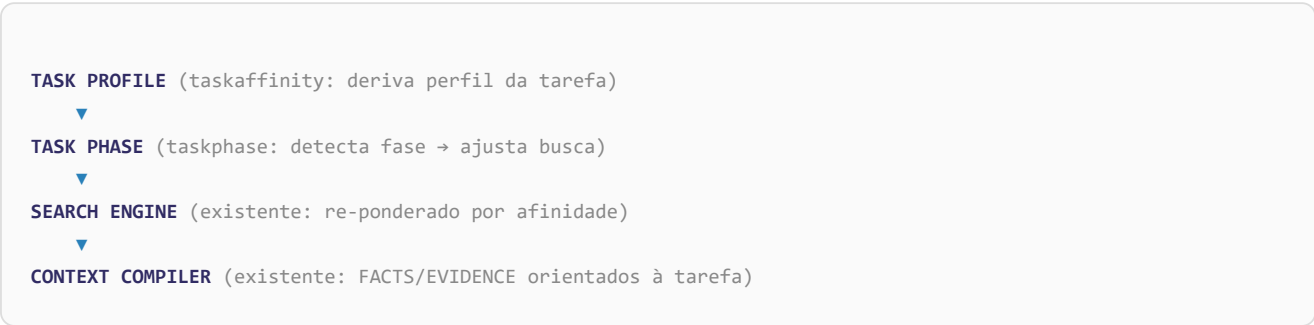
O **Task-Aware Search (TAS)** é a evolução mais recente da busca por entendimento. Ele adiciona duas camadas que conectam a busca ao **estado de implementação em curso**.

6.1 O Problema que Resolve

Antes do TAS, a busca sabia **o que** buscar (a intenção), mas não **o que estava sendo implementado**. O `task_affinity` — mencionado no ADR-035 §2.1 como "novo" — nunca tinha sido implementado.

O gap em duas frases: (1) A busca não sabe o que está sendo implementado — recebe a query crua, não o estado de implementação (stack, alvo, fase). (2) A busca não decide QUANDO usar entendimento profundo — no início (exploração) precisa de recall amplo, no meio (implementação) de entendimento profundo, no fim (verificação) de precisão cirúrgica.

6.2 Os Componentes



6.2.1 Task Profile (taskaffinity)

Perfil de tarefa derivado do estado de implementação real:

```
TASK:    implementar endpoint REST em Go
STACK:   go + chi + sqlite
TARGET:  internal/api/handlers.go
PHASE:   implementação (meio)
AFFINITY: [api, handlers, rest, chi, sqlite, project-patterns]
```

A busca **re-pondera** os resultados: o que casa com o perfil da tarefa sobe; o que é genérico desce. A fórmula:

```
boost = 0.15 × (termos_afinidade_no_resultado / total_afinidade) × confidence
score_final = score_ranking + boost
```

6.2.2 Task Phase (taskphase)

Detecta a fase da implementação e ajusta o modo de busca:

Fase	Modo de busca	Limit	Grafo	MinScore
Exploração (início)	recall amplo, entendimento leve	30	false	—
Implementação (meio)	entendimento profundo	20	true	0.15

Fase	Modo de busca	Limit	Grafo	MinScore
Verificação (fim)	precisão cirúrgica	10	false	0.30

Mudança-chave: o grafo (`EnableGraph`) só é ativado na fase de Implementação — é onde o "entendimento profundo" do padrão do projeto importa (arquivos relacionados, dependências entre módulos).

6.3 Performance Medida

O overhead do TAS é **desprezível** — medido em benchmarks:

Benchmark	Resultado	Critério	Status
DeriveProfile	3.0 μ s	< 1ms	✓
AffinityRerank (30 resultados)	86 μ s	< 500 μ s	✓
DetectPhase	7.6 ns	< 100 μ s	✓
TAS_Overhead (total)	65.8 μs	< 2ms	✓ 30x abaixo

Overhead total de ~66 microssegundos — 30x abaixo do teto de 2ms. A detecção de fase na casa dos nanossegundos preserva o núcleo determinístico e a soberania de CPU da plataforma.

6.4 Arquitetura de Implementação

Pacote	Arquivo	Função
<code>internal/taskaffinity</code>	taskaffinity.go	TaskProfile, ImplementaçãoState, DeriveProfile
<code>internal/taskaffinity</code>	rerank.go	AffinityRerank, computeAffinityBoost
<code>internal/taskphase</code>	taskphase.go	Phase, PhaseSignals, DetectPhase, PhaseSearchParams
<code>internal/taskaffinity</code>	buildcontext.go	TaskContext, BuildTaskContext
<code>internal/contextpipeline</code>	contextpipeline.go	Integração TAS no BuildContext
<code>internal/contextcompile</code>	contextcompile.go	EnrichWithProfile

7. Segurança e Governança

A segurança é inegociável. O Cosca opera com um modelo **fail-closed** — na dúvida, tranco.

7.1 Pipeline de Segurança

OBSERVE → READ-ONLY → PREVIEW → APPROVAL → WRITE → VERIFY → COMMIT

7.2 Mecanismos Enforced

Mecanismo	Comportamento real
Kernel Isolado / Proposals	Proposta = seal+hash ANTES de julgar; sem APPROVE → ErrDenied; hash ligado ao artefato (mutação → DENY)
Gate de transições com papel	approved/executed só don/admin; approver ≠ executor; append-only em gate.db
Classificador de operações	OplIrreversible → auto-block
Sandbox read-only	bwrap --ro-bind (workspace montado somente leitura)
Jail	bwrap com --clearenv; binário reexecutado de cópia em RAM; disco chmod -x
Secrets vault	AES-256-GCM; chave via HKDF-SHA256; arquivo 0600; valor nunca em JSON
Auth tokens	refresh-token com detecção de reuse → revoga TODAS as sessões
Trust boundary	Envelope() marca dados externos como não-instruções; Suspicious() detecta "ignore previous instructions"
Integrity/chain	Ed25519 + blake3; anti-hijack; git-anchor
Quality gate	build/test/vet/diff/secret scan/OSV → BLOCKED vs READY TO COMMIT

7.3 O Contrato de Segurança (o que NUNCA fazer)

- `internal/embed/cosca/` (o cérebro) é **read-only** — se tocar, re-assine a chain.
- A family chain é o sistema imune. Serve não sobe com chain desalinhada — é proteção, não bug.
- Nunca commitar segredos/runtime: `.env`, `secrets.db`, `auth_tokens.db`, `config.yaml`, `core.db`, `family_chain.dat` → fora do git.
- Artifacts de runtime (`knowledge.db`, `vector-*.db`, `graph.db`) são **regeneráveis** — não versionar.
- Apenas loopback — o serve escuta `127.0.0.1`. Auth obrigatória.
- Sandbox: Windows não tem bwrap → `COSCA_ALLOW_NO_ROOT=1` (opt-in). Código não-confiável só em WSL2+bwrap (zona Cofre).

7.4 A Lei do Cofre

O cofre NÃO abre para a nuvem. Identidade, memória, conhecimento e conversas são propriedade da família. A nuvem é um túnel que entra e sai sem ninguém ver. Chamadas remotas só com payload sanitizado, aprovação explícita do Don por sessão e trilha de auditoria. Resolve sem IA se puder → local primeiro (ollama) → nuvem só com aprovação.

8. Conclusão

O Cosca é um sistema de orquestração de agentes com um cérebro de conhecimento curado, protegido por uma family chain criptográfica e alimentado por uma busca semântica por entendimento de alta precisão.

De ponta a ponta, o fluxo é:

```
CHAIN (integridade/identidade)
  → BOOT (hardening + jail)
  → DESPERTAR (identidade determinística)
  → PIPELINE (memória → conhecimento → contexto → LLM → ferramentas → verificação)
  → BUSCA POR ENTENDIMENTO (FTS5 + vetor + grafo + re-rank + classificação)
  → TASK-AWARE SEARCH (afinidade + fase – ADR-045)
  → APRENDIZADO (chain assinada)
```

Os três pilares trabalham juntos:

- **A Family Chain** garante que o cérebro permaneça íntegro e a identidade verificável — a base de confiança.
- **A Busca por Entendimento** garante que o sistema entenda a intenção, não apenas a palavra — a base de inteligência.
- **O Pipeline** garante que cada etapa seja determinística, verificável e segura — a base de operação.

E com o **Task-Aware Search**, a busca agora também sabe o que está sendo implementado — trazendo o mínimo necessário que sustenta uma resposta correta, com overhead desprezível de ~66 microssegundos.

Lei da Família: Honestidade > Lealdade > Confiança; **Memória > Velocidade.** O Cosca não cresce por antecipação — cresce porque existe necessidade demonstrada.

Documento: Arquitetura de Ponta a Ponta — Chain, Busca Semântica e Pipeline
Versão: 1.0 | **Sistema:** Cosca v1.5.0
Classificação: CONFIDENCIAL — PROPRIEDADE DA FAMÍLIA
Licença: CoscaAI — v1.1.0 (2026-09-04)